

EXERCISE 2 : SQL foundations.

SQL Aggregate Function & Operators.

Matlabane Mngqay

Date: 1 May 2026

01: The Students Table.

1. List all distinct departments in the students table.

Expected columns: department

```
SELECT DISTINCT department  
FROM students;
```

2. Get the average age of students per department.

Expected columns: department, avg_age

```
SELECT department,  
       AVG(age)  
       AS avg_age  
FROM students  
GROUP BY department
```

Question 1 Output:

department
IT
HR
FINANCE

Question 2 output:

department	avg_age
IT	20.5
HR	22.0
FINANCE	23.0

3. Show departments with more than 1 student.

Expected columns: department, student_count.

```
SELECT department,  
COUNT(*) AS student_count  
FROM students  
GROUP BY department  
HAVING COUNT(*) > 1;
```

Outcome:

department	student_count
IT	2
HR	2

4. Get all students whose age is between 21 and 23.

Expected columns: student_id, name, age, department.

```
SELECT student_id,  
name,  
age,  
department  
FROM students  
WHERE age BETWEEN 21 AND 23;
```

Outcome

Student id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT IT
4	Diana	23	Finance
5	Eve	22	HR

5 List all students in the IT or HR department who are older than 21.

Expected columns: student_id, name, age, department

```
SELECT student_id,  
       name,  
       age,  
       department.
```

```
FROM students
```

```
WHERE department IN ('IT', 'HR')
```

```
AND age > 21;
```

outcome

student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

Q2: The Courses Table

6. Show table credits per department, only for departments with more than 5 total credits.

expected columns: Course_id, course_name, department, credits

```
SELECT department, SUM(credits) AS total_credits.
```

```
FROM courses
```

```
GROUP BY departments.
```

```
HAVING  
HAVING SUM(credits) > 5;
```

output:

department	total_credits
IT	11

7. List all courses that do not have 4 credits.

Expected columns: Course_id, course_name,
department, credits.

```
SELECT course_id, course_name, department, credits  
FROM courses  
WHERE credits != 4;
```

Outcome:

Course_id	Course_name	department	Credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

8. Show the top 3 courses by credits in descending order

Expected columns: Course_id, course_name, credits.

```
SELECT course_id,  
course_name,  
credits
```

```
FROM courses
```

```
ORDER BY credits DESC
```

```
LIMIT 3;
```

Outcome:

Course_id	Course_name	Credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

03: The enrollments table

enrollments.

9. Get the maximum, minimum and average grade across all expected columns: `course_id` `enrollment_count`.

```
SELECT MAX (grade) AS max_grade,  
       MIN (grade) AS min_grade,  
       AVG (grade) AS avg_grade  
FROM enrollments;
```

Outcome:

max_grade	min_grade	avg_grade
90	78	84.6

- 10 Count how many enrollments exist per course.
Expected columns: `course_id`, `enrollment_count`.

```
SELECT course_id,  
       COUNT (*) AS enrollment_count  
FROM enrollments  
GROUP BY course_id;
```

Outcome:

Count_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

Q4: The salaries table

11. Find total salary and total bonus per department.
Expected columns: department, total_salary, total_bonus

```
SELECT departments,  
       SUM(salary) AS total_salary,  
       SUM(bonus) AS total_bonus  
FROM salaries  
GROUP BY departments;
```

Outcome:

department	total_salary	total_bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

12. Show employees whose salary is above 55000
Expected columns: department, avg_salary

```
SELECT department,  
       AVG(salary) AS avg_salary  
FROM salaries  
GROUP BY department  
HAVING AVG(salary) > 55000;
```

Outcome:

department	avg_salary
IT	61000
Finance	70000

13 List employees whose salary plus bonus is greater than 60,000.

Expected columns: employee_id, name, salary, bonus
total_compensation.

```
SELECT employee_id,  
       name,  
       salary,  
       bonus,  
       salary + bonus AS total_compensation
```

```
FROM salaries
```

```
WHERE salary + bonus > 60000;
```

Outcome:

employee_id	name	salary	bonus	total_compensation
1	Tom	60000	5000	65000
3	Spice	70000	6000	76000
4	Tyre	62000	5500	67500

Q5: The projects Table.

14. Show total average budget per department, Only include departments with average budget above 70,000.

Expected columns: department, total_budget, avg_budget

```
SELECT department,  
       SUM (budget) AS total_budget,  
       AVG (budget) AS avg_budget
```

```
FROM projects
```

```
GROUP BY department
```

```
HAVING AVG (budget) > 70000 ;
```

Outcome:

department	total_budget	avg_budget
IT	270000	135000

15 List all projects with budgets between 50 000 and 120 000, excluding the Marketing department.
Expected columns: project_id, project_name, department, budget

SELECT project_id,
project_name,
department,
budget

FROM projects

WHERE budget BETWEEN 50000 AND 120000

AND department NOT = 'Marketing';

Outcome:

Project_id	Project_name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
5	HR Portal	HR	50000